

作业 6 exec 系统功能调用 和 shell 程序

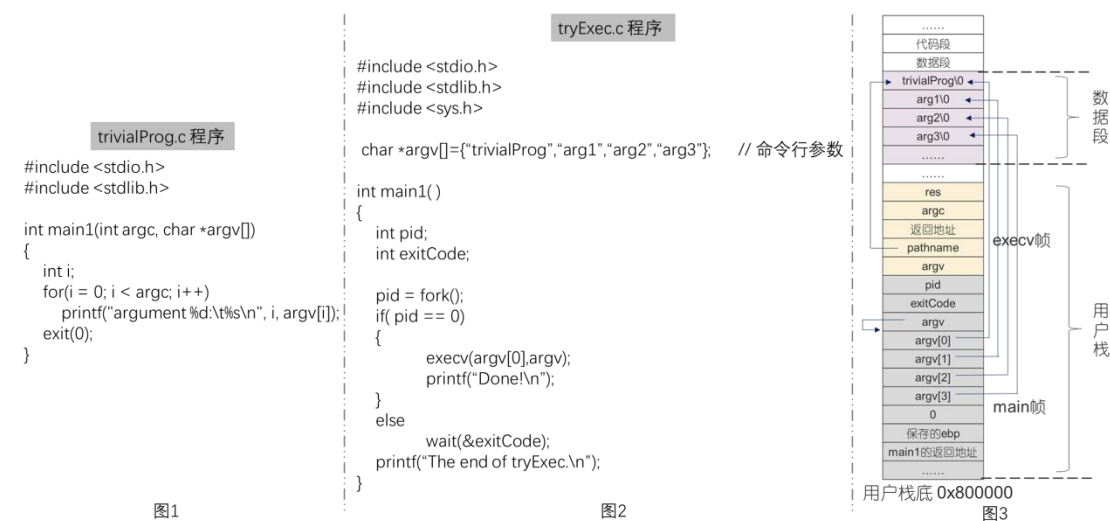
同济大学计算机系操作系统 国豪 2023 拔尖班 2023

姓名 毛星博

学号 2351520

一、解释这个程序的输出

命令行: #> tryExec



在为 oos 添加 tryExec.c 程序并重新编译后，在 bochs 虚拟机上运行程序：



- tryExec 程序的输出是：
 - argument0: trivialProg
 - argument1: arg1
 - argument2: arg2
 - argument3: arg3
 - The end of tryExec.
- 为什么会是这个输出？程序的执行流程如下：
 - 父进程启动：tryExec 程序开始运行。

- ② 创建子进程：调用 `fork()`。此时系统中有两个进程：父进程（`tryExec`）和子进程（`tryExec` 的副本）。
 - ③ 父进程行为：`fork()` 返回子进程的 PID（大于 0），进入 `else` 分支，调用 `wait(&exitCode)`。父进程在此处睡眠，暂停执行，等待直到子进程结束。
 - ④ 子进程行为：`fork()` 返回 0，进入 `if (pid == 0)` 分支，调用 `execv(argv[0], argv)`，即加载 `trivialProg`。`execv` 系统调用会用新的程序（`trivialProg`）的代码段、数据段和堆栈完全替换当前进程（子进程）的内存空间。一旦 `execv` 成功执行，子进程就不再是 `tryExec` 了，它变成了 `trivialProg`。因此，原 `tryExec.c` 中 `execv` 之后的语句 `printf("Done!\n")` 永远不会被执行，因为这部分代码在内存中已经被覆盖了。
 - ⑤ 新程序执行：子进程开始执行 `trivialProg` 的 `main1` 函数。它遍历参数并打印 `argument0` 到 `argument3`。执行 `exit(0)` 退出。
 - ⑥ 父进程恢复：子进程退出后，父进程的 `wait()` 函数返回。继续执行下一条语句：`printf("The end of tryExec.\n");`。
- 重点：
 - ① 当子进程执行 `execv(argv[0], argv)` 时，操作系统会把当前子进程的内存清空，加载新程序（这里是 `trivialProg`）的代码和数据。一旦 `exec` 成功，原来的代码（`tryExec.c` 中 `execv` 后面的代码）就完全消失了，不会再执行。子进程变成了 `trivialProg`。
 - ② 父进程调用 `wait`，会暂停运行，直到子进程结束（死掉）

二、修改 `tryExec.c` 程序，令 `tryExec` 进程执行系统调用时，用户栈

如上图 3 所示。

原来的代码，`argv` 定义在函数外部，它是全局变量，存储在数据段。

而图 3 显示 `argv` 数组位于 `main1` 函数的栈帧中。只有 `argv` 定义在 `main1` 函数内部，即它是局部变量时，才会存储在栈上。所以，只需把 `argv` 的定义从全局位置挪到 `main1` 函数里面。同时，由于 `UnixV6++` 系统中，变量定义的顺序为存放地址由高到低的顺序，所以需要根据图 3 的变量存放顺序来定义变量。

`tryExec.c` 程序如下所示：

```
int main1()
{
    char *argv[] = {"trivialProg", "arg1", "arg2", "arg3"};
    int exitCode;
    int pid;

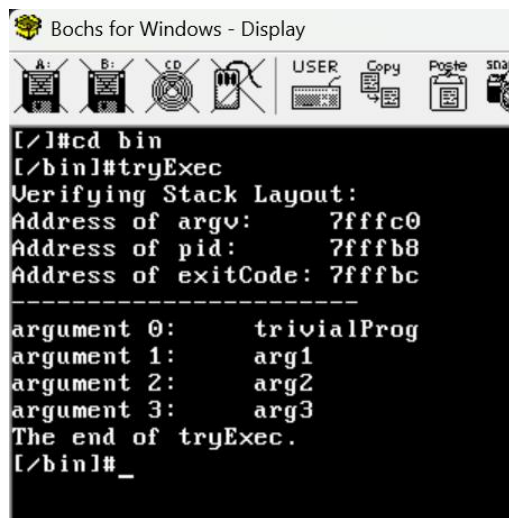
    pid = fork();
    if (pid == 0) // 子进程
```

```

    {
        execv("trivialProg", argv);
        printf("Done!\n");
    }
    else // 父进程，等待子进程结束
    {
        wait(&exitCode);
        printf("The end of tryExec.\n");
    }
}

```

验证：打印出 argv、pid 和 exitCode 的地址来验证，可以看到 pid、exitCode、argv 由低到高地址存放，证明了修改的正确性。



```

Bochs for Windows - Display
A: B: CD USER Copy Paste
[/]#cd bin
[/bin]#tryExec
Verifying Stack Layout:
Address of argv: 7fffc0
Address of pid: 7fffb8
Address of exitCode: 7ffbbc
-----
argument 0: trivialProg
argument 1: arg1
argument 2: arg2
argument 3: arg3
The end of tryExec.
[/bin]#_

```

三、修改 tryExec.c，写一个简版 shell 程序 myShell.c。myShell 进程睡眠等待用户输入命令行，驱动命令行指定的任意应用程序，并且为它提供所需的命令行参数。要求程序在 Unix V6++ 系统里调出来。

myShell.c 程序设计如下：

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys.h>

#define MAX_CMD_LEN 128
#define MAX_ARGS 20

```

```
int main1()
{
    char cmdLine[MAX_CMD_LEN];
    char *argv[MAX_ARGS];
    int argc;
    int pid, exitCode;
    int i;

    while (1)
    {
        printf("$ "); // 提示符
        gets(cmdLine); // 读取一行输入

        // 解析命令行
        argc = 0;
        int len = strlen(cmdLine);
        if (len == 0)
            continue;

        // 简单的空格分割
        int in_word = 0;
        for (i = 0; i < len; i++)
        {
            if (cmdLine[i] == ' ' || cmdLine[i] == '\t' || cmdLine[i] == '\n')
            {
                cmdLine[i] = '\0';
                in_word = 0;
            }
            else
            {
                if (in_word == 0)
                {
                    argv[argc++] = &cmdLine[i];
                    in_word = 1;
                }
            }
            if (argc >= MAX_ARGS - 1)
                break;
        }
        argv[argc] = 0; // NULL 结尾

        if (argc == 0)
            continue;
    }
}
```

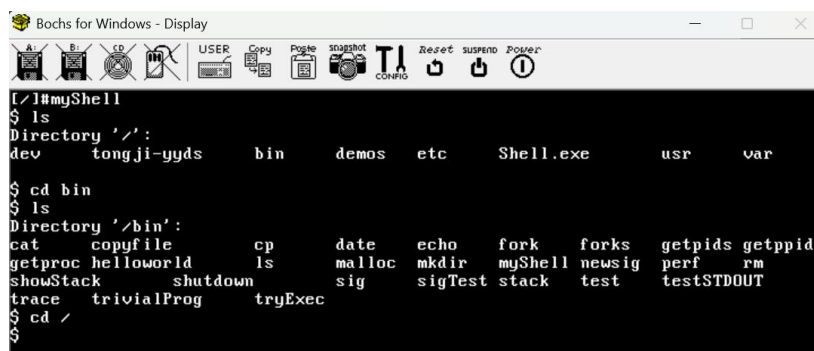
```

// 内部命令: logout
if (strcmp(argv[0], "logout") == 0)
    break;
// 内部命令: cd
else if (strcmp(argv[0], "cd") == 0)
{
    if (argc > 1)
        if (chdir(argv[1]) < 0)
            printf("cd: %s: No such file or directory\n", argv[1]);
        continue;
}

// 外部命令
pid = fork();
if (pid < 0)
    printf("fork failed\n");
else if (pid == 0)
    execv(argv[0], argv);
else
    // 父进程等待子进程结束
    wait(&exitCode);
}
return 0;
}

```

支持内部命令，包括 cd 命令和 logout 命令
也支持外部命令，这里演示 ls、trivialProg

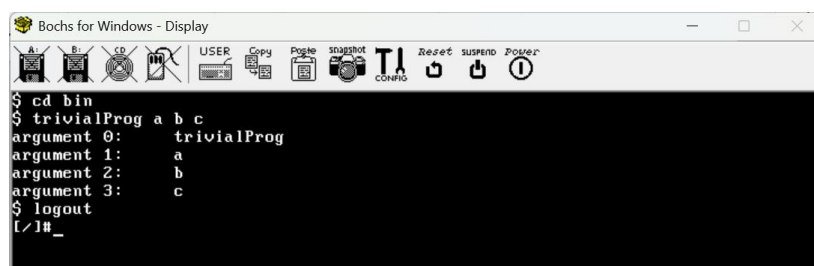


```

Bochs for Windows - Display
[1]myShell
$ ls
Directory '/':
dev    tongji-yyds    bin    demos    etc    Shell.exe    usr    var

$ cd bin
$ ls
Directory '/bin':
cat    copyfile    cp    date    echo    fork    forks    getpids    getppid
getproc    helloworld    ls    malloc    mkdir    myShell    newsig    perf    rm
showStack    shutdown    sig    sigTest    stack    test    testSTDOUT
trace    trivialProg    tryExec
$ cd /
$

```



```

Bochs for Windows - Display
$ cd bin
$ trivialProg a b c
argument 0:    trivialProg
argument 1:    a
argument 2:    b
argument 3:    c
$ logout
[1]#_

```

四、把只读数据段移到共享正文段里。给出你的设计，包括、可能不限于改动后的内存描述符 MemoryDescriptor 对象，exec 系统调用 和 MapToPageTable 函数。写设计，不强求调出来程序。

在当前的 UNIX V6++ 系统中，只读数据段(.rdata)被当作数据段的一部分处理，存储在每个进程私有的数据区域中。

当前系统中，进程的内存布局如下：

- 正文段(.text)：包含可执行代码，可以被多个进程共享
- 数据段(.data/.bss)：包含可读写数据和未初始化数据，每个进程私有
- 堆栈段：每个进程私有

把只读数据段移到共享正文段，设计方案如下：

1 改内存描述符 MemoryDescriptor

为了将只读数据段移到共享正文段，需要对 MemoryDescriptor 类进行以下修改：

- 新增成员变量：
 - m_RODataStartAddress: 只读数据段起始地址
 - m_RODataSize: 只读数据段长度
- 新增方法：
 - MapRODataEntrys(): 映射只读数据段到共享正文段
- 修改现有方法：
 - EstablishUserPageTable(): 增加对只读数据段的处理
 - MapEntry(): 修改以支持更多段类型

2 exec 系统调用设计

在 ProcessManager::Exec()函数中，需要进行以下修改来支持只读数据段的共享：

- PE 文件解析阶段：
 - 解析 PE 文件时识别.rdata 段
 - 分别记录.text 段和.rdata 段的信息
- 共享正文段处理：
 - 将.text 和.rdata 段合并为共享正文段
 - 在 Text 结构中增加对只读数据段的支持
- 内存映射阶段：
 - 修改 EstablishUserPageTable 调用，传入只读数据段信息
 - 在页表映射中将只读数据段映射到共享正文段

3 MapToPageTable 函数设计

修改 MemoryDescriptor::MapToPageTable()函数以支持只读数据段的共享映射：

- 页表映射逻辑：
 - 将.text 和.rdata 段都映射到共享正文段区域
 - 保持只读属性以确保数据安全

将.data/.bss/stack 段映射到进程私有数据区域

- 地址转换：
对于只读段(.text/.rdata)，使用共享正文段的物理地址基址
对于读写段(.data/.bss/stack)，使用进程私有数据区的物理地址基址
- 权限控制：
确保只读段在页表中被标记为只读
确保读写段在页表中被标记为可读写

4 PEParser 修改

需要修改 PEParser 类以正确识别和处理.rdata 段：

- 新增成员变量：
RODataAddress: 只读数据段虚拟地址
RODataSize: 只读数据段大小
- 修改 HeaderLoad 方法：
正确计算.text 段大小(到.rdata 段开始位置)
计算.rdata 段的起始地址和大小
调整.data 段的起始地址(从.bss 段开始)
- 修改 Relocate 方法：
确保在处理段时包含.rdata 段
正确设置.rdata 段的只读属性

通过以上设计方案，可以将只读数据段移到共享正文段，实现新的内存布局。